MLOPS FINAL PROJECT REPORT

FASHION DISCOVERY ENGINE

Important Links and Details -

- Name - Layaa Vishwakarma
- Roll No. - M25CSA017
- GitHub Link - https://github.com/Layaa-V/MLOps-LayaaVishwakarma-M25CSA017/tree/Final-Project
- Google Cloud Deployed Application link - http://34.93.246.183:8501/
- Live Monitoring (using Grafana) - http://35.244.56.146/
- Raw Dataset Link - https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images-dataset/code
- Demo Video link - MLOPS demo video.mov

Project Overview -

The Fashion Discovery Engine's core function is to allow a user to find relevant clothing products from a catalogue of 10,000 items either by typing a natural language description ("white sleeveless top", "floral summer tops") or by uploading a photograph of a clothing item. The system returns the three most visually and semantically similar products from the catalogue.

Set of MLOps practices used -

- Live A/B testing
- Real-time monitoring
- Comparison of Baseline and Finetuned model
- User feedback collection (feedback logging)
- Deployment using Google Cloud services

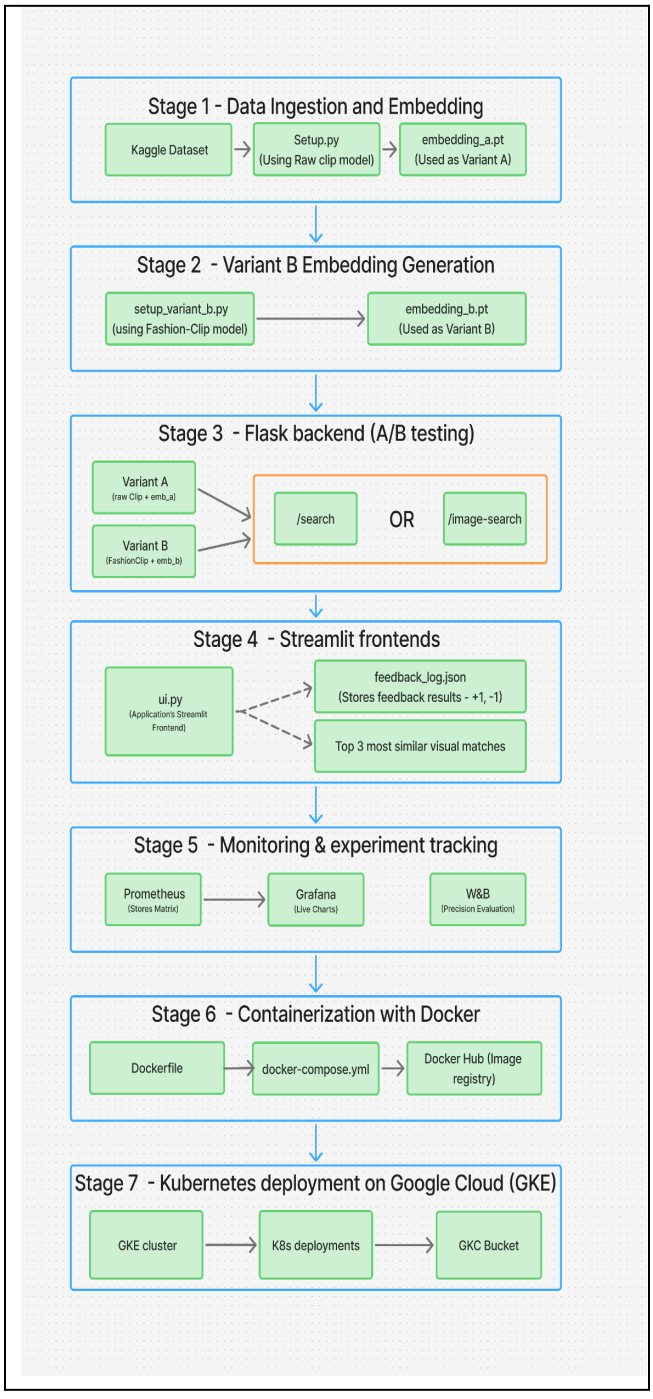
Technology Stack -

- OpenAI CLIP (openai/clip-vit-base-patch32) as Variant A
- FashionCLIP (patrickjohncyh/fashion-clip) as Variant B
- Flask (Backend)
- Prometheus-client (for metrics collection)
- Streamlit (Frontend)

- Grafana (live dashboard visualisation)
- Docker (containerisation) and Docker Hub (image registry)
- Kubernetes with GKE (cloud orchestration) and Google Cloud Storage (persistent data)

Pipeline Implementation -

As shown in the flowchart, the workflow was mainly divided into 7 stages, all of which are explained descriptively below -



Stage 1 - Data Ingestion and Embedding Generation

Our project utilizes the Kaggle Fashion Product Images Dataset, specifically filtered to the Apparel category. This focus ensures the search engine remains specialized, preventing non-clothing items from appearing in search results.

- The script ‘setup.py’ performs the entire data preparation pipeline. For each of the 10,000 selected products, the script does three things : resizes, runs the image through CLIP's image encoder, L2-normalises the vector.
- After Using the CLIP Vision Transformer (ViT-B/32), each image is converted into a 512-dimensional vector that captures its visual essence.
- All 10,000 vectors are stacked into a single PyTorch tensor of shape (10000, 512) and saved as ‘embeddings_a.pt.’

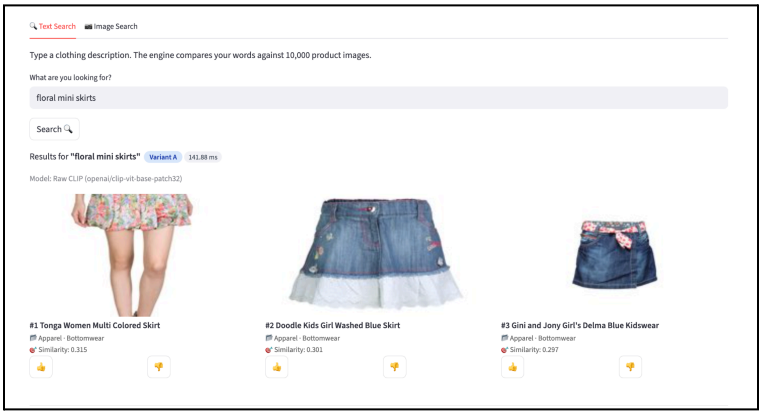
This model produces embeddings in a shared text-image space, meaning that a text description of a product and the product's image, when encoded by CLIP, will produce vectors that are geometrically close to each other if they describe the same thing.

FLOWCHART
DEPICTING THE PIPELINE

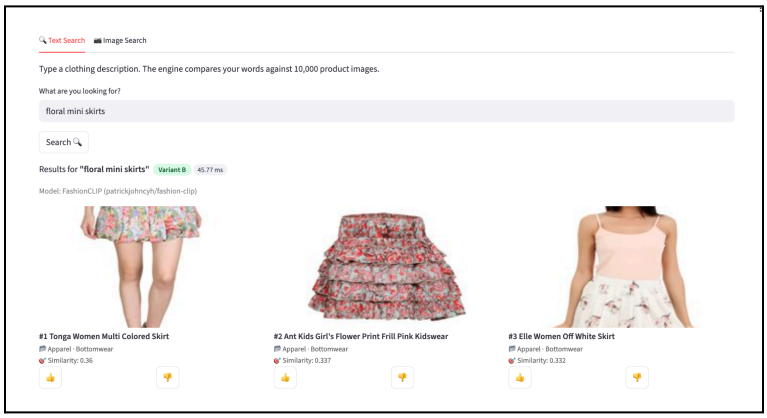
Stage 2 - Variant B Embedding (A/B Testing)

The A/B testing was implemented by using 2 different models for Variant A and Variant B, where variant B was a fine-tuned version w.r.t the dataset (clothing and fashion).

- The script ‘setup_variant_b.py’ re-processes all 10,000 product images through FashionCLIP's image encoder and saves the result as ‘embeddings_b.pt’
- FashionCLIP is built on the same ViT-B/32 architecture as raw CLIP but fine-tuned on 800,000 fashion-specific image-text pairs, hence has meaningfully better performance on fashion queries.
- **Observation :** This experiment measures a real capability difference between the two models rather than an implementation artifact. Variant B significantly showed higher ‘Thumb-Up’ feedback than Variant A.



Results from Variant A



Results from Variant B

The above images show results from both variants A and B, run on the query ‘Floral mini Skirts’. We can clearly see that the results of Variant B are much more semantically aligned with the search query.

Stage 3 - Flask Backend (Feedback logging)

- The Flask backend serves as the application's central processing system, orchestrating model management, real-time search, and system observability. It runs through the file ‘app.py’. At startup, the system loads the CLIP models and embed tensors into RAM, followed by encoding queries (test phrase) like "blue dress".

- The backend manages three primary customer facing endpoints: /search, /image-search and /feedback.
    - **/search** - accepts a text query as a URL parameter.
    - **/image-search -** accepts a base64-encoded image in the POST body, decodes it to a PIL image.
    - **/feedback -** accepts a JSON body containing the query, the clicked product name, the rating (1 for thumbs-up, -1 for thumbs-down), and the variant that served the result.
- Each query is randomly assigned to either Variant A or Variant B in a 50/50 split. The query is encoded and compared against the stored embeddings of respective model, runs a cosine similarity search against the query's embedding tensor and returns the top 3 results with their product images, names, categories, similarity scores, and a label identifying which variant served the request. .
- /feedback appends this feedback logs permanently to 'feedback_log.json' on disk and uploads the updated file to Google Cloud Storage so the feedback survives container restarts.
- The backend exposes a dedicated metrics endpoint that Prometheus scrapes every 15 seconds. This provides real-time visibility into search latency, average similarity scores, and model health,which is then presented in a presentable form through grafana.

Stage 4 - Streamlit Frontend:

The streamlit UI provides a seamless search experience through these special features:

- **Clear choice selection:** Through 2 different provided buttons, users can select very comfortably whether they want to search the product using textual or visual input.
- **Rendered image outputs:** The outputs are provided in a very clear distinctive format, showing their similarity score and name as well.
- **Feedback buttons:** The outputs are provided in a very clear distinctive format, showing their similarity score and name as well.

Stage 5 - Real Time Monitoring and Experiment Tracking(Grafana)

To track the application in real time, we paired Prometheus with Grafana. Prometheus acts as the silent data collector, checking the backend every 15 seconds to record metrics like search speed and model accuracy. Grafana then turns that raw data into a live visual dashboard. This helps us to see at a glance:

- **Model Performance comparison:** A direct comparison of similarity scores between the raw CLIP and the fine-tuned FashionCLIP.
- **System Reliability:** Tracking "p95 Latency" to ensure that 95% of users receive their results in under one second.
- **User Satisfaction:** Real-time charts of thumbs-up vs. thumbs-down feedback, categorized by which model served the result.
- **Overall crowd presence:** How many times the website has been reached out in a given span of time.
- **Current Engagement:** How many customers are live on the website at the current moment.

A snapshot of live working Grafana Dashboard

Stage 6 - Containerisation with Docker:

To ensure the application is portable and reliable, we used Docker to package the entire system into a single, cohesive environment. This ensures the code runs identically whether it's on a local laptop or a production server.

The backend is built using a two stage Docker pattern to optimize speed and stability:

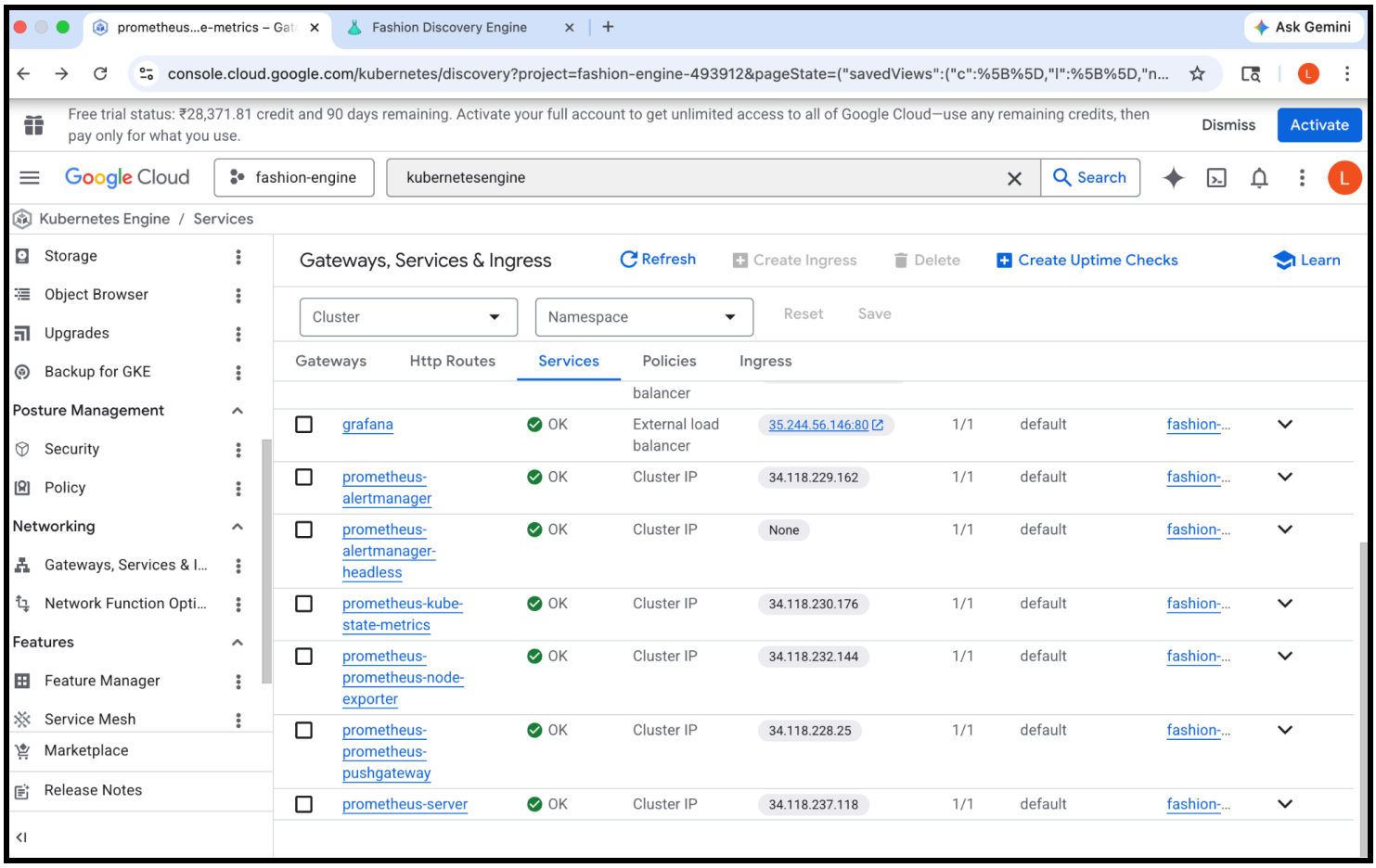
- We first download the AI model weights directly into the Docker cache. By using a "download-only" approach, we avoid loading the massive models into memory during the build process, which prevents crashes on smaller servers.
- This stage pulls in the pre downloaded models and installs only the necessary tools for running the app. We use Gunicorn to manage the server, specifically limiting it to two workers to carefully manage the system's memory usage.
- The ‘docker-compose.yml’ file orchestrates four containers together for local testing: the Flask backend, the Streamlit customer frontend, Prometheus, and Grafana. This tool acts as a conductor, ensuring they can all communicate with one another.
- Docker Compose mounts feedback_log.json as a volume so feedback persists even when the backend container is restarted without rebuilding.
- The final built images are then pushed to Docker Hub, which acts as a distributor for kubernetes, which then pulls these images and forms a live working container (a pod).

Stage 7 - Kubernetes Deployment on Google Cloud:

This transition, from docker to a cloud-native architecture (Google Cloud here) moves the project from a local prototype to a resilient, professional service. The system is hosted on GKE (Google Kubernetes Engine) in the Mumbai region to ensure low-latency access for users. We utilized Kubernetes manifests to automate the system’s health and uptime.
‘backend-deployment.yaml’ defines a deployment.

- The cluster maintains two active backend pods, so that if one fails, Kubernetes instantly replaces it to prevent downtime.
- Readiness probes ensure traffic only reaches a pod after its AI models are fully loaded, while Liveness probes automatically restart unresponsive containers.
- ‘backend-service.yaml’ defines a ClusterIP service that gives the backend a stable internal IP address regardless of which pod is currently running or what IP they have been assigned.
- ‘frontend-service.yaml’ defines a LoadBalancer service that provides a public IP address from Google Cloud's infrastructure so users on the internet can reach the Streamlit UI.
- Google Cloud Storage (GCS) serves as the persistent backing store for ‘feedback_log.json’. When the Flask backend receives a feedback POST request, it writes to the local file on disk and then uploads the updated file to the GCS bucket. Hence, feedback is never lost.

- We use rolling updates to replace pods one by one. This ensures at least one version of the app is always live during a new deployment.



A snapshot of Google Cloud Console showing public IP addresses for the app and monitoring dashboard (grafana)

Another trial towards project building (another version) -

The initial reason for storing feedback_logs.json in such a non-destroyable way was that I attempted to fine-tune variant B with real time feedback (CI/CD implementation). In this version, variant B was also a raw CLIP model, which would be fine tuned with time using the feedbacks. But, since the feedback count was too low (as it was a personal model not enough feedback could be collected), hence fine tuning did not make any difference, rather created a bias.

Hence, that approach was dropped and I opted for Fashion-CLIP as my new Variant B.